Software Requirements Specification (SRS)

Project: Real-Time Chat Application (Socket Programming)

Version: 1.0

Team No: 5

Authors: Amoghaa R, Vaibhav R, P S Harshavardhan, Pramath Ganesh Hegde

Date: 13-09-2026

Status: Final

Team

Name	SRN	Role / Focus Area
Amoghaa R	PES2UG24CS625	Testing & Documentation
Vaibhav R	PES2UG24CS664	Server / Socket Backend
P S Harshavardhan	PES2UG25CS817	Authentication & Database
Pramath Ganesh Hegde	PES2UG24CS648	Frontend / UI

Revision history

Version	Date	Author	Change summary	Approval
1.0	13-09-2026	Team 5 (Amoghaa R, Vaibhav R, P S Harshavardhan, Pramath Ganesh Hegde)	Initial SRS for Chat Application (PS #5)	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			

Table of Contents

1. Introduction
2. Overall description
3. External interface requirements
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. System models and diagrams
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for a real-time Chat Application built using socket programming (Node.js + WebSockets, or Python Flask-SocketIO). It defines functional and non-functional requirements, interfaces, and verification criteria for the development and evaluation of the system.

1.2 Scope

Covers user registration and authentication, one-to-one and group real-time messaging over WebSockets, presence and notifications, file/media sharing, message history and search, and basic administration/moderation. Excludes voice/video calling, end-to-end encryption key management beyond transport security, and third-party push-notification gateway internals.

1.3 Audience

Developers, QA Evaluators, Course Instructors, and Project Reviewers.

1.4 Definitions

Acronyms: WS (WebSocket), WSS (WebSocket Secure), REST, JWT, UI, API, TLS, XSS, DoS, RTM, FR (Functional Requirement), NFR (Non-Functional Requirement).

2. Overall description

2.1 Product perspective

The Chat Application is a client-server system in which a browser or app client connects to a socket server (Node.js + Socket.IO/WebSockets, or Python Flask-SocketIO) that relays real-time events between clients and persists conversation data in a backing database (e.g., MongoDB/PostgreSQL/SQLite).

2.2 Major product functions (detailed)

- User registration, login, and profile management
- One-to-one real-time text messaging
- Group chat creation and real-time group messaging
- Presence indication (online/offline/last-seen) and typing indicators
- File and image sharing within a conversation
- Persistent chat history with retrieval on reconnect and search
- Notifications for new/missed messages
- Administration: reporting, suspension, and system-wide announcements

2.3 User roles and characteristics

- User: registered chat participant; expects low-latency delivery and a simple, responsive UI.
- Group Admin: a user with elevated rights within a specific group (add/remove members).
- System Admin: platform-level role responsible for moderation and announcements.

2.4 Operating environment

Server: Node.js runtime with Socket.IO (or Python 3 with Flask-SocketIO), deployed behind a reverse proxy supporting WebSocket upgrade. Client: modern web browser (desktop and mobile) with WebSocket support. Database: document or relational store for users, messages, and groups.

2.5 Constraints

Must use socket-based (not polling-only) real-time transport; single technology stack choice (Node.js+WebSockets OR Python Flask-SocketIO) for the implementation; limited team size and academic-semester timeline; attachment storage limited to local/server storage rather than a dedicated CDN.

3. External interface requirements

3.1 User interfaces

A responsive web UI with a conversation list, active chat window, message composer with typing indicator, and group management screens. Accessible color contrast and keyboard navigation for message composition.

3.2 Hardware interfaces

None beyond standard client device network interface, camera/file picker for attachments, and speaker for optional notification sound.

3.3 Software interfaces

- WebSocket/Socket.IO event channel between client and server for real-time messaging, presence, and typing events.
- REST API for authentication, conversation/group management, message history retrieval, and file upload.
- Database interface for persisting users, groups, messages, and attachment metadata.

3.4 Communications

WSS (WebSocket Secure) and HTTPS enforced for all client-server traffic; automatic client reconnect with exponential backoff on connection loss and re-sync of missed messages on reconnect.

[Overall project provides 20 FRs, 5 NFRs, 2 security objectives, and 5 security requirements — see Sections 4 and 5.]

4. System features (detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow CHAT-F-###.

4.1 Authentication & User Management

Description: Register, authenticate, and manage a user's identity and profile.

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-001	The system shall allow a new user to register with a unique username/email and password.	Functional	High	User	AC-001: Duplicate username/email rejected; valid registration creates account. Test: TC-Auth-01	Password hashed before storage
CHAT-F-002	The system shall authenticate a user's credentials and establish an authenticated WebSocket session.	Functional	High	User	AC-002: Valid credentials open a session; invalid ones are rejected with an error. Test: TC-Auth-02	Depends on CHAT-SR-003

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-003	The system shall allow a user to log out and invalidate the active session/token.	Functional	Medium	User	AC-003: Logged-out session cannot send/receive further events. Test: TC-Auth-03	

4.2 One-to-One Messaging

Description: Real-time direct messaging between two users over WebSockets.

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-010	The system shall deliver a text message from a sender to an online recipient in real time via WebSocket.	Functional	High	User	AC-010: Recipient receives message within NFR-001 latency target. Test: TC-Msg-01	Core socket feature
CHAT-F-011	The system shall persist every sent message so it can be delivered when the recipient reconnects.	Functional	High	User	AC-011: Offline recipient sees the message on next login. Test: TC-Msg-02	Requires database
CHAT-F-012	The system shall show delivery and read receipts (sent/delivered/read) for a direct message.	Functional	Medium	User	AC-012: Status icon updates as message state changes. Test: TC-Msg-03	
CHAT-F-013	The system shall show a typing indicator to the recipient while the sender is composing a message.	Functional	Low	User	AC-013: Indicator appears within 1s of typing and clears after inactivity. Test: TC-Msg-04	Extends CHAT-F-010

4.3 Group Chat

Description: Multi-user chat rooms with membership management.

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-020	The system shall allow a user to create a group chat with a name and an initial member list.	Functional	High	User	AC-020: New group appears for all initial members. Test: TC-Grp-01	
CHAT-F-021	The system shall allow a group admin to add or remove members from the group.	Functional	Medium	Group Admin	AC-021: Membership change reflected for all members in real time. Test: TC-Grp-02	Authorization check required
CHAT-F-022	The system shall broadcast a group message to all current members in real time.	Functional	High	User	AC-022: All online members receive the message; offline members receive it on reconnect. Test: TC-Grp-03	Includes CHAT-F-011 logic
CHAT-F-023	The system shall allow a user to leave a group they belong to.	Functional	Low	User	AC-023: User removed from member list and stops receiving new group events. Test: TC-Grp-04	

4.4 Presence & Notifications

Description: Indicate user availability and surface new-message alerts.

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-030	The system shall display a user's online/offline status and last-seen timestamp to their contacts.	Functional	Medium	User	AC-030: Status updates within 5s of a connection state change. Test: TC-Pres-01	
CHAT-F-031	The system shall notify a user of missed messages when they reconnect after being offline.	Functional	Medium	User	AC-031: Missed-message summary/badge shown on reconnect. Test: TC-Pres-02	
CHAT-F-032	The system shall display an unread-message count per conversation.	Functional	Low	User	AC-032: Count increments on new message and clears when conversation is opened. Test: TC-Pres-03	

4.5 File & Media Sharing

Description: Attach and view files/images within a conversation.

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-040	The system shall allow a user to upload and share a file or image within a direct or group conversation.	Functional	Medium	User	AC-040: Recipient(s) can view/download the shared attachment. Test: TC-File-01	
CHAT-F-041	The system shall validate uploaded attachments against a maximum size and an allowed file-type list before storing them.	Functional	High	Security	AC-041: Oversized or disallowed file types are rejected with an error. Test: TC-File-02	Supports CHAT-SR-005

4.6 Message History & Search

Description: Retain and retrieve past conversation content.

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-050	The system shall persist chat history and allow a user to scroll back through past messages in a conversation.	Functional	High	User	AC-050: Historical messages load on opening a conversation. Test: TC-Hist-01	
CHAT-F-051	The system shall allow a user to delete their own message within a configurable time window.	Functional	Low	User	AC-051: Deleted message is replaced by a placeholder for other participants. Test: TC-Hist-02	

4.7 Administration & Moderation

Description: Platform-level controls for abuse handling and announcements.

Req ID	Requirement (shall...)	Type	Priority	Source/ Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
CHAT-F-060	The system shall allow a System Admin to suspend or ban a reported user account.	Functional	Medium	System Admin	AC-060: Suspended user cannot log in or send messages. Test: TC-Adm-01	Includes CHAT-SR verification
CHAT-F-061	The system shall allow a System Admin to broadcast a system-wide announcement to all connected users.	Functional	Low	System Admin	AC-061: All online users receive the announcement banner/message. Test: TC-Adm-02	

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans. IDs CHAT-NF-###.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
CHAT-NF-001	End-to-end message delivery latency (send to receive) shall be ≤200ms for 95% of messages under a load of 100 concurrent connections.	Performance	High	Load test shows p95 ≤200ms. Test: TC-Perf-01
CHAT-NF-002	The system shall support at least 100 concurrent WebSocket connections on a single server instance without message loss or noticeable degradation in delivery latency.	Scalability	Medium	Load test with 100 simulated clients shows no dropped messages and latency stays within NFR-001's target. Test: TC-Scale-01
CHAT-NF-003	The system shall provide 99.5% monthly availability, excluding scheduled maintenance, with automatic client reconnect within 5 seconds of a dropped connection.	Reliability	High	Uptime monitoring reports; reconnect timing test. Test: TC-Rel-01
CHAT-NF-004	The chat UI shall be responsive on both desktop and mobile browsers, allowing a first message to be sent within 3 clicks/taps after login.	Usability	Medium	Usability walkthrough on 2 screen sizes. Test: TC-UX-01
CHAT-NF-005	The codebase shall be modular with a documented REST/WebSocket API and at least 80% unit test coverage on server-side logic.	Maintainability	Medium	Coverage report ≥80%; API docs reviewed. Test: TC-Maint-01

5.1 Security

5.1.1 Security Objectives

- Protect user credentials and message content from unauthorized access or interception (confidentiality).
- Ensure only authenticated users can establish a WebSocket session and access conversations they are a member of (authentication & access control).

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
CHAT-SR-001	The system shall enforce TLS/WSS (WebSocket Secure) for all client-server communication.	Security	High	Network scan confirms no plaintext WS/HTTP endpoints. Test: TC-Sec-01

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
CHAT-SR-002	The system shall hash passwords using a strong algorithm (e.g., bcrypt/argon2) and never store or log them in plaintext.	Security	High	Database review + log audit finds no plaintext passwords. Test: TC-Sec-02
CHAT-SR-003	The system shall require a valid authentication token to establish a WebSocket handshake and shall reject unauthenticated connection attempts.	Security	High	Connection attempt without a token is refused. Test: TC-Sec-03
CHAT-SR-004	The system shall verify server-side that a user is a member of a conversation/group before delivering or accepting messages for it.	Security	High	Attempt to access a foreign conversation is denied. Test: TC-Sec-04
CHAT-SR-005	The system shall apply rate limiting on message-send events per connection to mitigate spam and denial-of-service via socket flooding.	Security	Medium	Burst of messages beyond threshold is throttled/rejected. Test: TC-Sec-05

6. Quality attributes & Acceptance tests

Exit criteria for acceptance: all High-priority functional requirements implemented and verified, no critical NFR failures, and the RTM shows all mapped test cases passed.

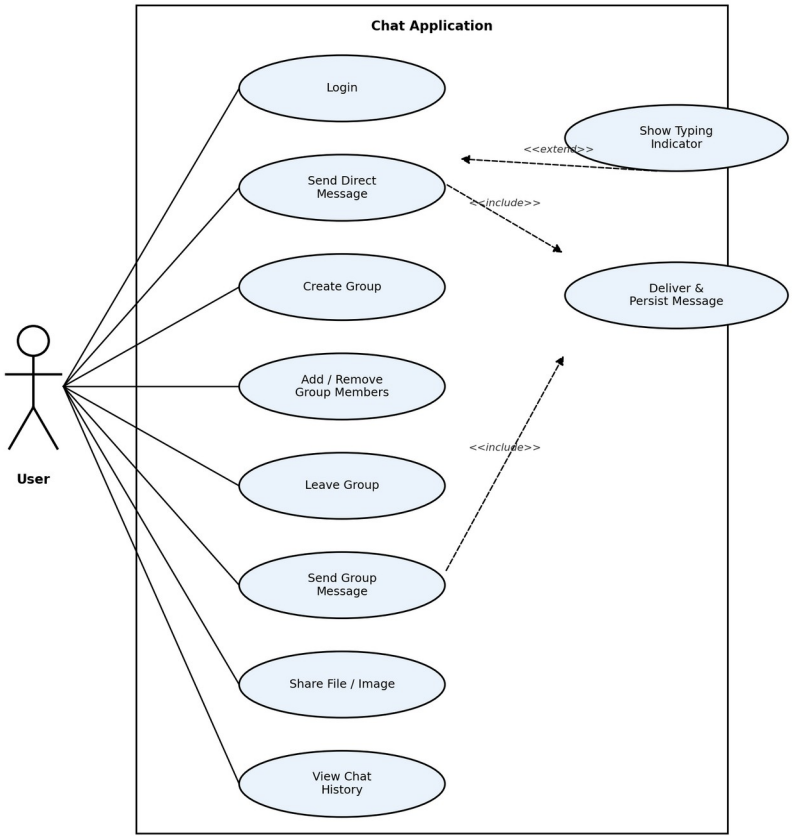
Acceptance test suites: Authentication, Direct Messaging, Group Chat, Presence & Notifications, File Sharing, History & Search, Administration, Performance, and Security.

7. System models and diagrams

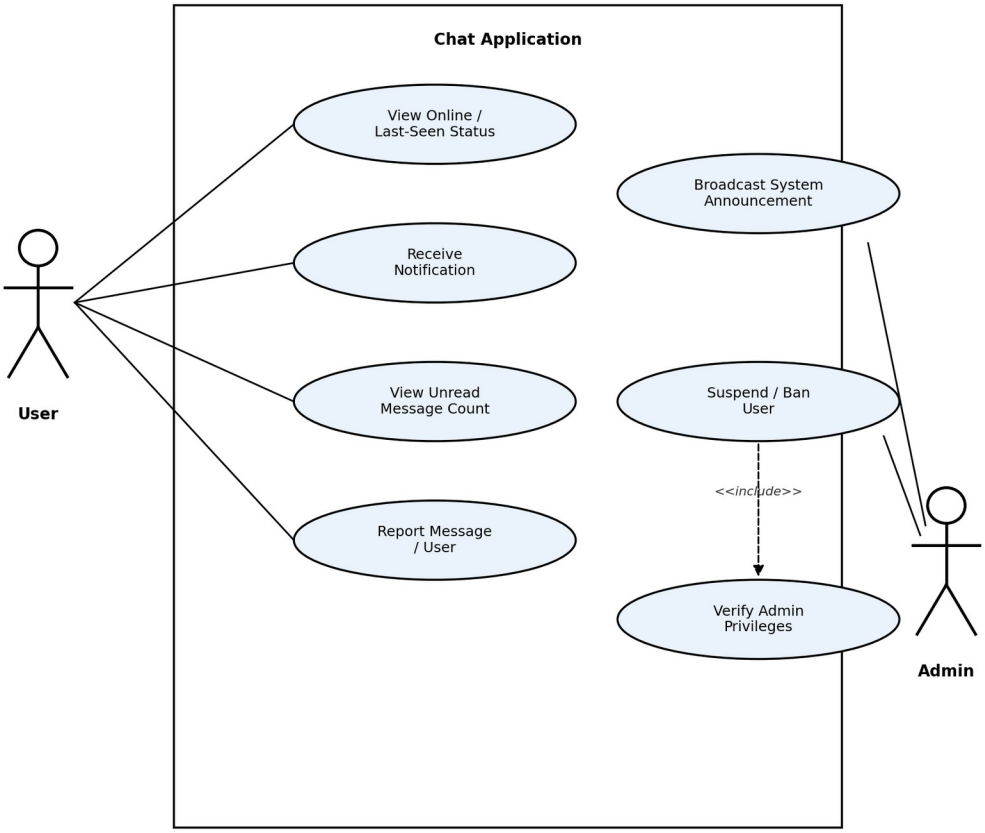
7.1 UML Use-Case Diagrams

Two use-case diagrams are provided below, covering (A) core messaging and group-chat behavior and (B) presence, notifications, and administration.

Use-Case Diagram A: Messaging & Group Chat



Use-Case Diagram B: Presence, Notifications & Administration



8. Requirements Traceability Matrix (RTM)

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
CHAT-F-001	Register user	4.1	AuthModule	TC-Auth-01	N	
CHAT-F-002	Authenticate & open session	4.1	AuthModule	TC-Auth-02	N	
CHAT-F-003	Logout / invalidate session	4.1	AuthModule	TC-Auth-03	N	
CHAT-F-010	Send direct message	4.2	MessagingModule	TC-Msg-01	N	
CHAT-F-011	Persist message for offline delivery	4.2	MessagingModule / DB	TC-Msg-02	N	
CHAT-F-012	Delivery & read receipts	4.2	MessagingModule	TC-Msg-03	N	
CHAT-F-013	Typing indicator	4.2	MessagingModule	TC-Msg-04	N	
CHAT-F-020	Create group	4.3	GroupModule	TC-Grp-01	N	
CHAT-F-021	Add/remove group members	4.3	GroupModule	TC-Grp-02	N	
CHAT-F-022	Broadcast group message	4.3	GroupModule	TC-Grp-03	N	
CHAT-F-023	Leave group	4.3	GroupModule	TC-Grp-04	N	
CHAT-F-030	Presence status	4.4	PresenceModule	TC-Pres-01	N	
CHAT-F-031	Missed-message notification on reconnect	4.4	PresenceModule	TC-Pres-02	N	
CHAT-F-032	Unread message count	4.4	PresenceModule	TC-Pres-03	N	
CHAT-F-040	File/image sharing	4.5	UploadModule	TC-File-01	N	
CHAT-F-041	Validate attachment size/type	4.5	UploadModule	TC-File-02	N	
CHAT-F-050	Persist & retrieve chat history	4.6	HistoryModule	TC-Hist-01	N	
CHAT-F-051	Delete own message	4.6	HistoryModule	TC-Hist-02	N	
CHAT-F-060	Suspend/ban user	4.7	AdminModule	TC-Adm-01	N	
CHAT-F-061	Broadcast system announcement	4.7	AdminModule	TC-Adm-02	N	
CHAT-NF-001	Message latency target	5	MessagingModule	TC-Perf-01	N	
CHAT-NF-002	Concurrent connection support	5	ServerInfra	TC-Scale-01	N	
CHAT-NF-003	Availability & reconnect	5	ServerInfra	TC-Rel-01	N	

Req ID	Requirement (short)	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
CHAT-NF-004	Responsive UI / usability	5	WebUI	TC-UX-01	N	
CHAT-NF-005	Modular code & test coverage	5	Codebase-wide / CI	TC-Maint-01	N	
CHAT-SR-001	TLS/WSS enforced	5.1.2	Infra/Networking	TC-Sec-01	N	
CHAT-SR-002	Password hashing	5.1.2	AuthModule	TC-Sec-02	N	
CHAT-SR-003	Authenticated WS handshake	5.1.2	AuthModule	TC-Sec-03	N	
CHAT-SR-004	Server-side membership check	5.1.2	MessagingModule / GroupModule	TC-Sec-04	N	
CHAT-SR-005	Rate limiting on message send	5.1.2	MessagingModule	TC-Sec-05	N	